

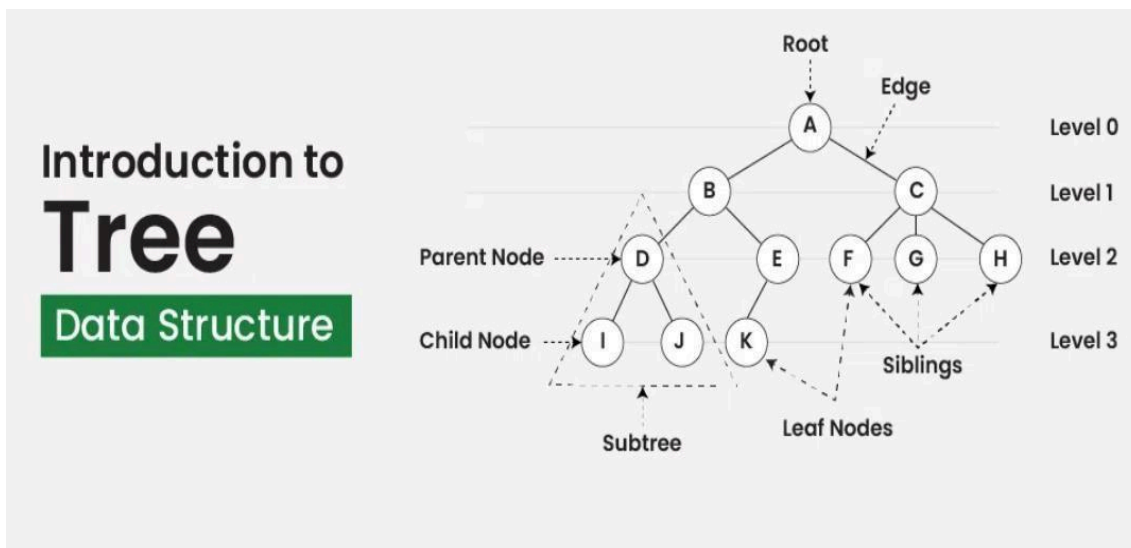


Introduction to Tree

What is a Tree?

A **tree** is a non-linear hierarchical data structure made up of nodes, where:

- One node is designated as the **root**.
- Each node may have **zero or more child nodes**.
- There is exactly one path between the root and any other node in the tree.
- Trees are used to represent data with a hierarchical relationship, such as file systems, organizational charts, and XML/HTML data.



Basic Terminology

Term	Description
Root	The topmost node of the tree.
Parent	A node that has one or more child nodes.
Child	A node that descends from another node (parent).
Leaf	A node that has no children.
Edge	A link between two nodes.
Siblings	Nodes that share the same parent.
Level	The distance from the root node (root is level 0).
Height of tree	The longest path from the root to a leaf.
Subtree	A tree consisting of a node and its descendants.
Degree	The number of children a node has.

Binary Tree (BT) – The General Case

A **Binary Tree** is simply a tree data structure where **each node can have at most two children**:

- Left child
- Right child

Characteristics:

- No specific order between nodes.
- Used for representing hierarchical data (like expression trees, syntax trees, etc.).
- Can be of different types: **Full**, **Complete**, **Perfect**, **balanced**, etc.

Example:

```
10
 / \
5  20
 / \
2  30
```

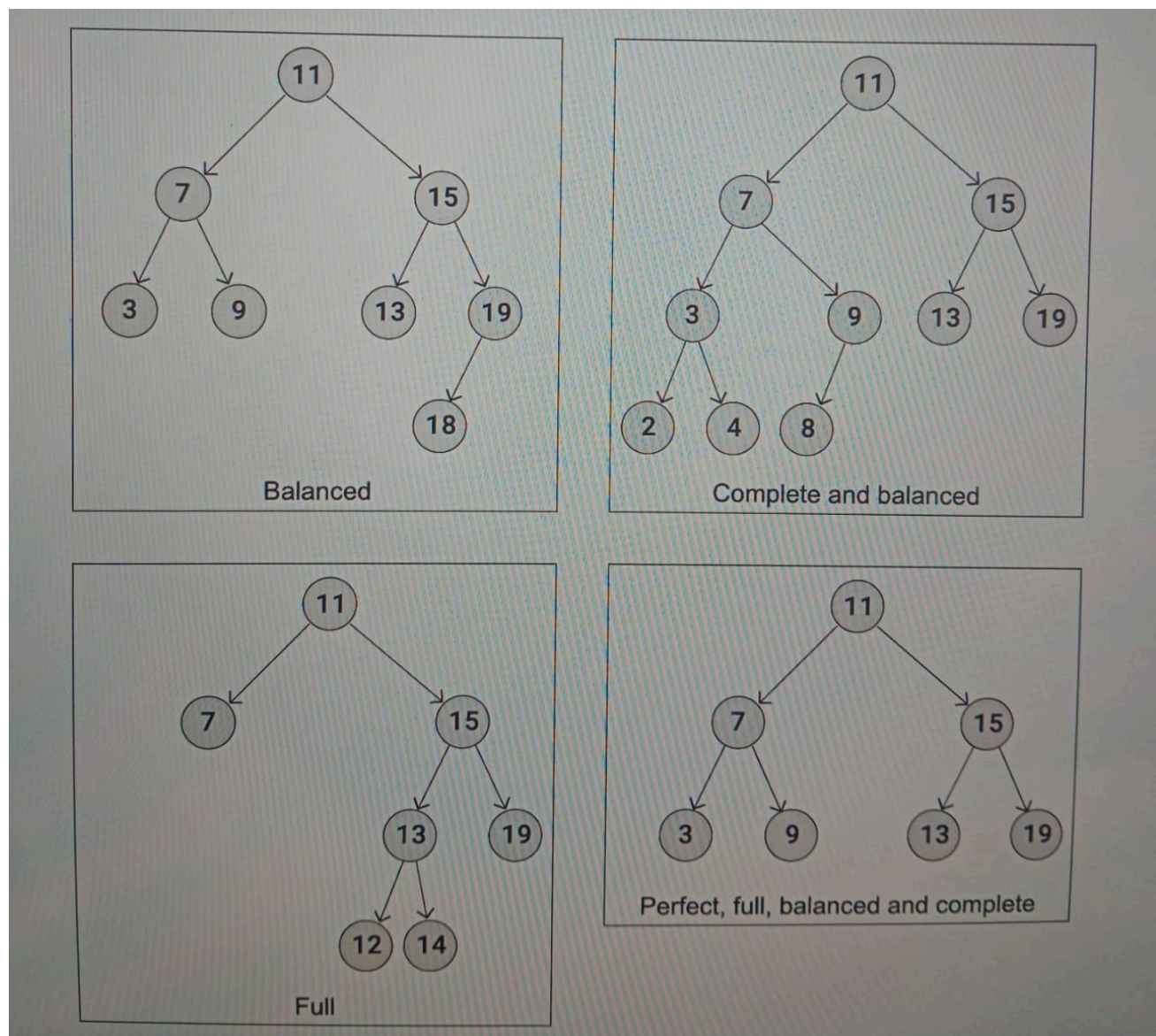
Types of Binary Trees

A **balanced Binary Tree** has at most 1 in difference between its left and right subtree heights, for each node in the tree.

A **complete Binary Tree** has all levels full of nodes, except the last level, which is can also be full, or filled from left to right. The properties of a complete Binary Tree means it is also balanced.

A **full Binary Tree** is a kind of tree where each node has either 0 or 2 child nodes.

A **perfect Binary Tree** has all leaf nodes on the same level, which means that all levels are full of nodes, and all internal nodes have two child nodes. The properties of a perfect Binary Tree means it is also full, balanced, and complete.



What is a Binary Search Tree (BST)?

A **Binary Search Tree (BST)** is a type of binary tree that follows a specific ordering rule:

- **Left Subtree** contains nodes with values **less than** the parent node.
- **Right Subtree** contains nodes with values **greater than** the parent node.
- This rule applies **recursively** to all nodes in the tree.

Characteristics:

- Left < Root < Right
- In-order traversal gives **sorted order** of elements.
- Efficient searching, insertion, and deletion (average $O(\log n)$).

Example: 50, 30, 20, 40, 70, 60, 80



In this BST:

- All values in the left subtree of 50 are less than 50.
- All values in the right subtree of 50 are greater than 50.

BSTs are widely used for searching, insertion, and deletion operations because they offer **efficient performance (average time complexity: $O(\log n)$)**, especially when balanced.

Key Differences

Feature	Binary Tree (BT)	Binary Search Tree (BST)
Definition	Each node has ≤ 2 children	A binary tree with an ordering property
Order of nodes	No rule	Left < Root < Right
Purpose	Used for structure/hierarchy	Used for fast search, insert, delete
Traversal output (inorder)	Any order	Sorted order
Example usage	Expression Tree, Huffman Tree	Searching, Sorting, Databases (indexes)
Search time	$O(n)$	$O(\log n)$ average, $O(n)$ worst
Insertion rule	Anywhere	Must follow BST property

Tree traversals

Inorder Traversal (Left, Root, Right)

Preorder Traversal (Root, Left, Right)

Postorder Traversal (Left, Right, Root)

Example Tree

Let's take this binary tree: 10, 5, 3, 7, 20, 15, 25

```
      10      inorder: 3 5 7 10 15 20 25
     /  \    pre: 10 5 3 7 20 15 25
    5    20  post: 3 7 5 15 25 20 10
   /  \  /  \
  3    7 15 25
```

1 Inorder Traversal (Left → Root → Right)

Algorithm:

1. Traverse left subtree
2. Visit root
3. Traverse right subtree

Order for above tree: 3 → 5 → 7 → 10 → 15 → 20 → 25

In BSTs, Inorder traversal always gives sorted order

2 Preorder Traversal (Root → Left → Right)

Algorithm:

1. Visit root
2. Traverse left subtree
3. Traverse right subtree

Order for above tree: **10** → **5** → **3** → **7** → **20** → **15** → **25**

Used for creating a copy of a tree or building expression trees

3 Postorder Traversal (Left → Right → Root)

Algorithm:

1. Traverse left subtree
2. Traverse right subtree
3. Visit root

Order for above tree: **3** → **7** → **5** → **15** → **25** → **20** → **10**

Used for deleting a tree (since we delete children before root).

implement a Binary Search Tree with insert and traversal

class Node:

```
def __init__(self, data):
```

```
self.data = data
```

```
self.left = None
```

```
self.right = None
```

```
root = None
```

```
# Function to insert a new node into the BST
```

```
def insert(root, data):
```

```
    if root is None:
```

```
        return Node(data)
```

```
    if data < root.data:
```

```
        root.left = insert(root.left, data)
```

```
    elif data > root.data:
```

```
        root.right = insert(root.right, data)
```

```
    return root
```

```
# Function for inorder traversal (Left, Root, Right)
```

```
def inorder(root):
```

```
    if root is not None:
```

```
        inorder(root.left)
```

```
        print(root.data, end=" ")
```



```
inorder(root.right)
```

Function to search for a value in the BST

```
def search(root, key):
```

```
    if root is None:
```

```
        return False
```

```
    if root.data == key:
```

```
        return True
```

```
    elif key < root.data:
```

```
        return search(root.left, key)
```

```
    else:
```

```
        return search(root.right, key)
```

```
values = [50, 30, 20, 40, 70, 60, 80]
```

Insert nodes into BST

```
for val in values:
```

```
    root = insert(root, val)
```

Inorder traversal

```
print("Inorder Traversal (Left, Root, Right):", end=" ")
```

```
inorder(root)
```

```
print()
```

Search for a value

```
key = 60
```

```
if search(root, key):
```

```
    print(f"{key} is present in the BST.")
```

```
else:
```

```
    print(f"{key} is not present in the BST.")
```

Output:

Inorder Traversal (Left, Root, Right): 20 30 40 50 60 70 80

60 is present in the BST.

BST in inorder, preorder and postorder.

```
class Node:
```

```
    def __init__(self, data):
```

```
self.data = data
```

```
self.left = None
```

```
self.right = None
```

```
root = None
```

```
# Function to insert a new node into the BST
```

```
def insert(root, data):
```

```
    if root is None:
```

```
        return Node(data)
```

```
    if data < root.data:
```

```
        root.left = insert(root.left, data)
```

```
    else:
```

```
        root.right = insert(root.right, data)
```

```
    return root
```

```
# Inorder Traversal (Left, Root, Right)
```

```
def inorder(root):
```

```
if root is not None:
```

```
    inorder(root.left)
```

```
    print(root.data, end=" ")
```

```
    inorder(root.right)
```

```
# Preorder Traversal (Root, Left, Right)
```

```
def preorder(root):
```

```
    if root is not None:
```

```
        print(root.data, end=" ")
```

```
        preorder(root.left)
```

```
        preorder(root.right)
```

```
# Postorder Traversal (Left, Right, Root)
```

```
def postorder(root):
```

```
    if root is not None:
```

```
        postorder(root.left)
```

```
postorder(root.right)
```

```
print(root.data, end=" ")
```

```
values = [50, 30, 20, 40, 70, 60, 80]
```

```
# Insert nodes into BST
```

```
for val in values:
```

```
    root = insert(root, val)
```

```
print("Inorder Traversal (Left, Root, Right): ", end="")
```

```
inorder(root)
```

```
print()
```

```
print("Preorder Traversal (Root, Left, Right): ", end="")
```

```
preorder(root)
```

```
print()
```

```
print("Postorder Traversal (Left, Right, Root): ", end="")
```

```
postorder(root)
```

```
print()
```

Output:

Inorder Traversal (Left, Root, Right): 20 30 40 50 60 70 80

Preorder Traversal (Root, Left, Right): 50 30 20 40 70 60 80

Postorder Traversal (Left, Right, Root): 20 40 30 60 80 70 50

implement a Binary Search Tree with deletion

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
root = None
```

```
def insert(root,data):
```

```
    if root is None:
```

```
        return Node(data)
```

```
if data < root.data:
```

```
    root.left = insert(root.left,data)
```

```
else:
```

```
    root.right = insert(root.right,data)
```

```
return root
```

```
def inorder(root):
```

```
    if root is not None:
```

```
        inorder(root.left)
```

```
        print(root.data,end = " ")
```

```
        inorder(root.right)
```

```
def findmin(node):
```

```
    current = node
```

```
    while current.left is not None:
```

```
        current = current.left
```

```
    return current
```

```

def delete(root,key): #20

    if root is None:

        return root

    if (key < root.data):

        root.left = delete(root.left,key)

    elif(key > root.data):

        root.right = delete(root.right,key)

    else:

        # No child in leaf node

        if root.left == None and root.right == None:

            return None

        #One child

        elif root.left is None:

            return root.right

        elif root.right is None:

            return root.left

```


#Two children – find inorder successor

```
temp = findmin(root.right)
```

```
root.data = temp.data
```

```
root.right = delete(root.right,temp.data)
```

```
return root
```

```
value = [50,30,20,40,70,60,80]
```

```
for val in value:
```

```
    root = insert(root,val)
```

```
print("Inorder traversal (Left,Root,Right): ")
```

```
inorder(root)
```

Delete a node

```
key = 30
```

```
print(f"\nDeleting {key}...")
```

```
root = delete(root, key)
```

```
print("After deletInorder traversal :")
```

```
inorder(root)
```

Output:

Inorder traversal (Left,Root,Right):

20 30 40 50 60 70 80

Deleting 30...

After deletion inorder traversal :

20 40 50 60 70 80